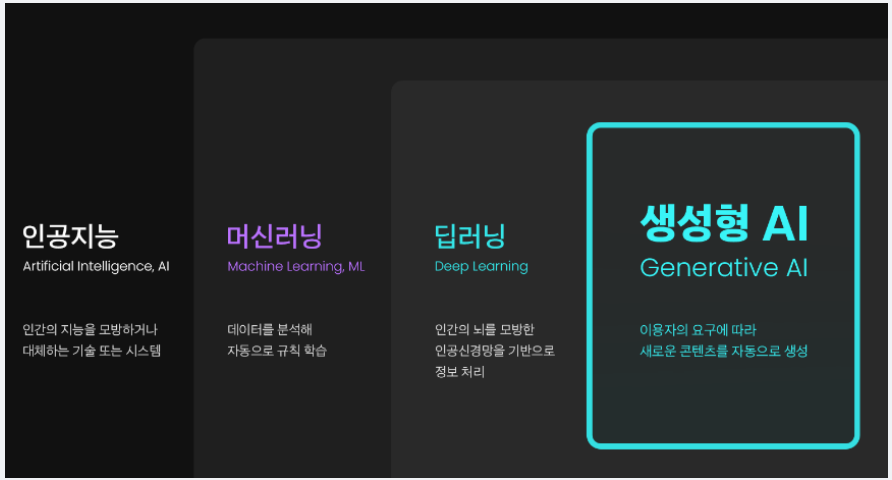
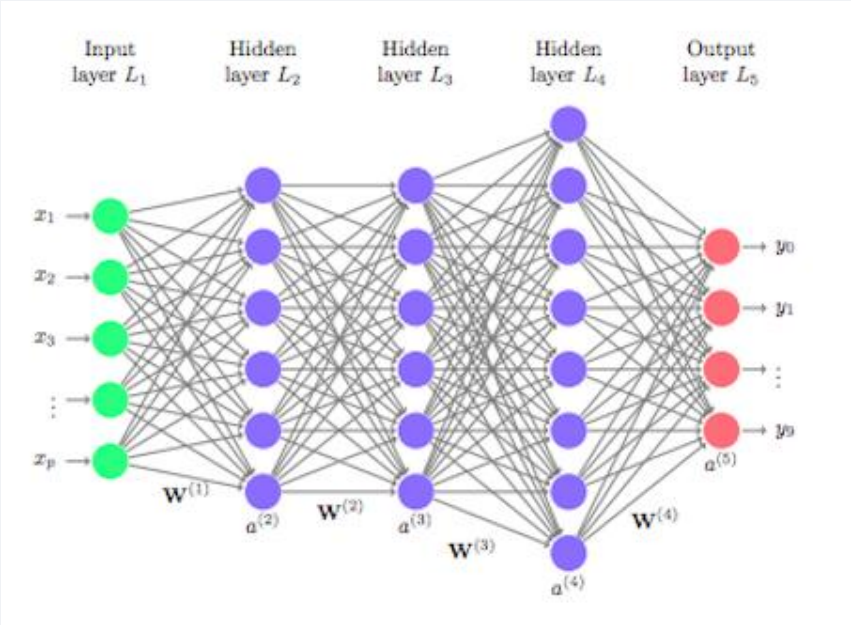
딥러닝: 개념과 작동 원리

이종혁(경희대 미디어학과)

참고: 딥 러닝을 이용한 자연어 처리 입문 (wikidocs.net/book/2155)

인공지능, 머신러닝, 딥러닝의 관계





https://medium.com/@venkatesh.t.16072001/what-is-deep-learning-how-it-changes-the-world-f1dfb8b7e0ef
http://tobetong.com/?p=9393
https://news.skhynix.co.kr/post/all-around-ai-1

딥러닝의 역사 — 주요 이정표

시기	사건	의의
1943	McCulloch & Pitts 인공 뉴런	최초의 수학적 뉴런 모델
1957	Rosenblatt 퍼셉트론	초기 인공 신경망 등장
1969	Minsky XOR 문제 지적	단층 퍼셉트론의 한계 → AI 겨울
1986	Hinton 역전파 알고리즘	다층 신경망 학습 가능 → 부활
1989	LeCun CNN(LeNet) 제안	이미지 인식용 합성곱 신경망
1997	Hochreiter LSTM	장기 의존성 문제 해결
2006	Hinton Deep Belief Network	딥러닝 부흥의 신호탄
2012	AlexNet ImageNet 우승	GPU + 빅데이터 → CNN 대중화
2014	GAN (Goodfellow)	생성 모델 혁명
2017	Transformer (Google)	Attention Is All You Need → NLP 혁신
2018	BERT (Google)	사전학습 기반 양방향 언어 모델
2020~	GPT-3, ChatGPT, GPT-4	초거대 언어모델(LLM) 시대

참고: wikidocs.net/263871

퍼셉트론 (Perceptron) — 인공 신경망의 시작

퍼셉트론이란?

Frank Rosenblatt(1957)가 제안한 초기 인공 신경망

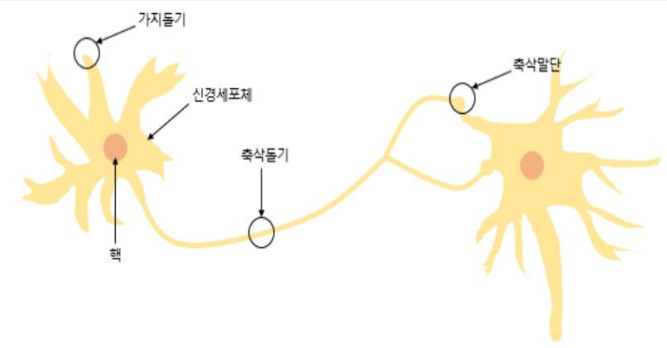
뇌의 뉴런을 모방한 수학적 모델:
 뉴런: 신호를 받아 임계치를 넘으면 다음 뉴런에 전달
 퍼셉트론: 입력값 x 가중치의 합이 임계치를 넘으면 1 출력

수학적 구조:
 z = w1*x1 + w2*x2 + ... + b (선형 변환)
 y = f(z) (활성화 함수 적용)

한계: XOR 문제를 풀 수 없음
 → 직선 하나로 구분 불가능한 문제
 → 1969년 Minsky가 지적 → AI 겨울

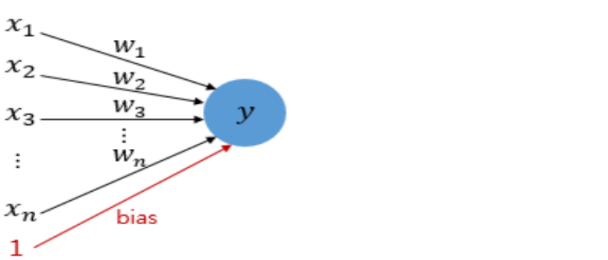
생물학적 뉴런과의 비유

가지돌기 → 입력값 (x1, x2, ...)
시냅스 강도 → 가중치 (w1, w2, ...)
세포체 → 가중합 계산 (z = sum)
축삭돌기 → 출력 (y = 0 또는 1)
임계치 → 편향 (bias, b)



핵심 용어

가중치(Weight, W): 각 입력의 중요도
편향(Bias, b): 임계치 역할의 상수
활성화 함수: 비선형 변환 적용
 → 계단 함수(Step Function) 사용
학습: 가중치를 조정하는 과정



$$if \sum_i^n w_i x_i + b \geq 0 \rightarrow y = 1$$

$$if \sum_i^n w_i x_i + b < 0 \rightarrow y = 0$$

그림 참고: wikidocs.net/book/2155

다층 퍼셉트론 (MLP) — XOR 문제 해결과 딥러닝의 탄생

1 단층 퍼셉트론의 한계

직선 하나로 구분할 수 없는 문제(XOR)를 해결하지 못함

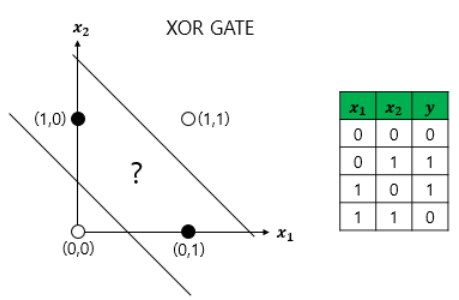
→ 1969년 Minsky & Papert가 증명

2 다층 퍼셉트론(MLP)

입력층과 출력층 사이에 은닉층(hidden layer) 추가 → 비선형 문제 해결 가능

은닉층이 2개 이상 → 심층신경망(DNN)

DNN을 학습시키는 것 → 딥러닝(Deep Learning)



x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

3 층(Layer)의 종류

입력층(Input Layer): 데이터를 받는 층

은닉층(Hidden Layer): 특징을 추출하는 층

출력층(Output Layer): 최종 결과를 내는 층

전결합층(FC) = 밀집층(Dense): 모든 뉴런이 이전 층의 모든 뉴런과 연결

4 피드포워드 신경망(FFNN)

입력층 → 은닉층 → 출력층 방향으로만 연산이 진행되는 구조

예외: RNN은 은닉층 출력이 다시 은닉층 입력으로 순환(recurrent)

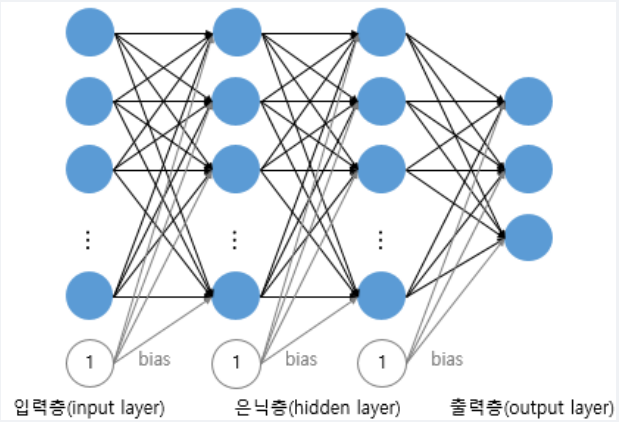
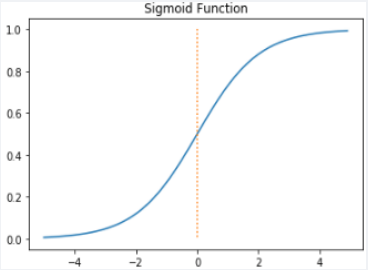


그림 참고: wikidocs.net/images/page/24958

활성화 함수 (Activation Function)

가중치 연산 후 비선형 변환을 적용하여 출력 값을 결정하는 함수. 활성화 함수가 없으면 아무리 층을 쌓아도 결국 하나의 선형 변환과 같아짐 → 비선형성이 핵심.



1 시그모이드 (Sigmoid)

출력: 0 ~ 1 사이 값

용도: 이진 분류 출력층에서 주로 사용

문제점: 은닉층에서 사용하면 기울기 소실(Gradient Vanishing) 발생
→ 출력이 0이나 1에 가까워지면 미분값이 거의 0이 됨

$$\frac{e^x}{e^x + 1}$$

2 소프트맥스 (Softmax)

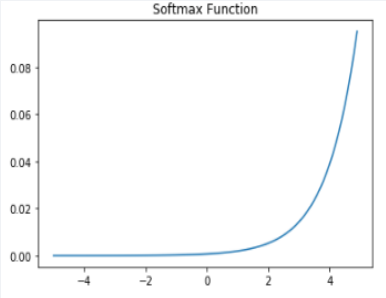
출력: 각 클래스의 확률 (합 = 1)

용도: 다중 클래스 분류 출력층

예: 개(0.7), 고양이(0.2), 새(0.1)
→ 가장 높은 확률의 클래스 선택

시그모이드의 다중 클래스 버전

$$\frac{e^{x_i}}{\sum_{j=1} e^{x_j}}$$



3 렐루 (ReLU)

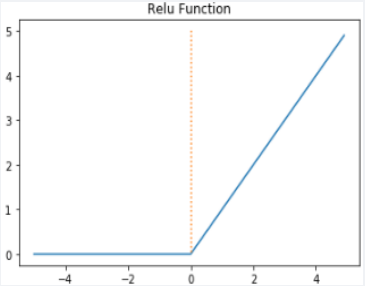
f(x) = max(0, x)
음수 → 0, 양수 → 그대로 반환

용도: 은닉층에서 가장 많이 사용

장점: 기울기 소실 문제 완화
계산이 매우 빠름

변형: Leaky ReLU, GELU, Swish 등

$$max(0, x)$$



학습 방법 — 지도학습, 비지도학습, 자기지도학습

1 지도학습 (Supervised Learning)

레이블(정답)이 있는 데이터로 학습
입력과 정답을 쌍으로 제공 → 패턴 학습

예: 이미지 분류, 감정 분석, 객체 탐지
"이 사진은 고양이" → 고양이를 학습

2 비지도학습 (Unsupervised Learning)

레이블 없는 데이터로 학습
데이터의 구조나 패턴을 스스로 찾음

예: 클러스터링, 차원 축소
비슷한 뉴스끼리 자동으로 묶기

3 자기지도학습 (Self-Supervised Learning)

데이터 자체에서 지도 신호를 생성
대규모 레이블 없는 데이터 활용 가능

예: BERT(마스크된 단어 예측)
GPT(다음 단어 예측)

4 강화학습 (Reinforcement Learning)

환경과 상호작용하며 보상을 최대화
시행착오를 통해 최적의 행동 전략 학습

예: 게임 AI, 로봇 제어
ChatGPT의 RLHF(인간 피드백 강화학습)

데이터 분리와 학습 단위

데이터 분리 (Splitting Data)

학습용 (Training Data)

모델 학습에 사용. 전체의 70~80%

비유: 문제 풀기

검증용 (Validation Data)

학습 중간에 성능 점검. 약 10~15%

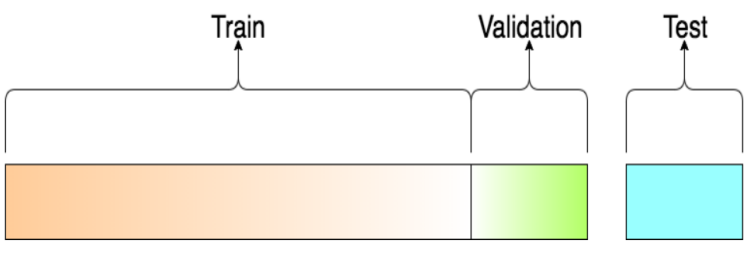
비유: 모의고사

평가용 (Test Data)

최종 성능 측정. 약 10~15%

비유: 실제 시험

학습 과정에 절대 사용하지 않음



학습(training) 관련 용어

배치 크기 (Batch Size)

한 번에 모델에 넣는 데이터 수

예: 전체 2000개, 배치 200 → 10개 배치

이터레이션 (Iteration = Step)

1 에포크를 끝내기 위한 배치 처리 횟수

위 예시에서 이터레이션 = 10

매 이터레이션마다 가중치 업데이트

에포크 (Epoch)

전체 데이터를 1회 순전파+역전파 완료

에포크 10 = 전체 데이터로 10번 학습

에포크가 너무 많으면 과적합 위험

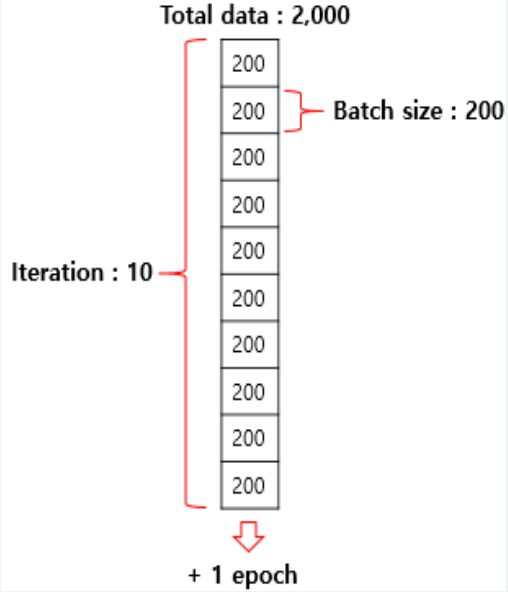


그림 참고: wikidocs.net/images/page/36033/batchandepochiteration.PNG

순전파 (Forward Propagation) — 예측 만들기

1 입력 받기
데이터가 입력층에 들어감

→

2 선형 변환
z = W*x + b
(가중치 곱 + 편향)

→

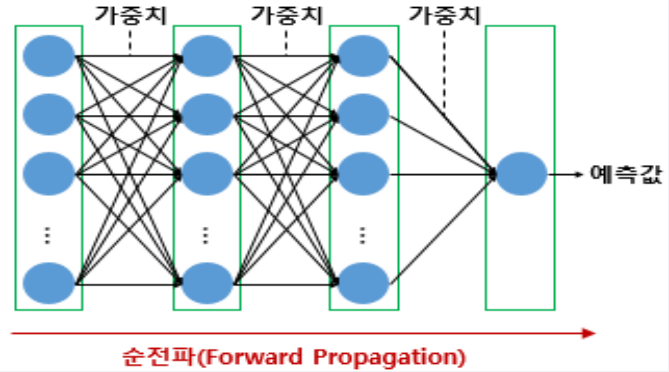
3 활성화 함수
h = f(z)
(비선형 변환)

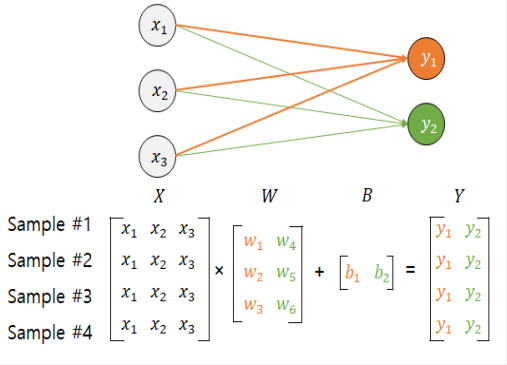
→ **반복 (은닉층 통과)** →

4 출력층에서 예측 생성
분류: Softmax로 확률 출력 | 회귀: 그대로 수치 출력

순전파의 핵심

입력층 → 은닉층 → 출력층 방향으로 한 방향 연산
각 층에서 "선형 변환(Wx+b) → 활성화 함수" 반복
최종 출력층에서 예측값(y-hat) 생성 → 이 예측값과 실제값의 차이를 손실함수로 계산
학습 초기에는 가중치가 무작위이므로 예측이 엉터리
→ 역전파로 가중치를 조정해야 함





역전파 (Backpropagation) — 가중치 업데이트

역전파란?

출력층 → 은닉층 → 입력층 방향으로 거꾸로 계산하며
가중치(W)와 편향(b)을 업데이트하는 과정

핵심 원리: 미분의 연쇄법칙 (Chain Rule)
"최종 오차가 각 가중치에 얼마나 영향을 미쳤는가?"
를 역방향으로 추적하며 계산

비유:
시험(순전파)을 보고 틀린 문제(손실)를 확인한 뒤
어디서 틀렸는지 거꾸로 추적(역전파)하며
공부 방법(가중치)을 수정하는 과정

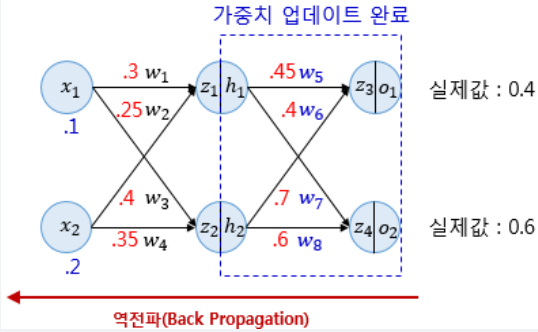
수학적 표현:
dL/dW = dL/dy * dy/dz * dz/dW
(연쇄법칙으로 각 가중치에 대한 기울기 계산)
→ 이 기울기 방향으로 가중치를 조정

미분 기호 정리

dy/dx: 일반 미분 (순간 변화율, 접선 기울기)
partial y/partial x: 편미분
(다변수 함수에서 x에 대한 변화율)
nabla: 그래디언트(기울기 벡터)
(모든 변수에 대한 편미분 모음)
$\nabla L = (\partial L/\partial w1, \partial L/\partial w2, \partial L/\partial w3)$

기울기 소실 문제

시그모이드 미분값: h*(1-h)
→ 출력이 0이나 1에 가까우면
미분값이 거의 0
→ 연쇄법칙으로 계속 곱하면
기하급수적으로 작아짐
→ 층이 깊을수록 심각
→ 해결: ReLU 사용, 잔차 연결 등



w_1 업데이트: 아래 미분 값이 0에 가까워짐

$$\frac{\partial E_{\text{total}}}{\partial w_1} = \frac{\partial E_{o1}}{\partial w_1} + \frac{\partial E_{o2}}{\partial w_1}$$

$$\frac{\partial E_{o1}}{\partial w_1} = \frac{\partial E_{o1}}{\partial o_1} \cdot \frac{\partial o_1}{\partial z_3} \cdot \frac{\partial z_3}{\partial h_1} \cdot \frac{\partial h_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_1}$$

$$\frac{\partial E_{o2}}{\partial w_1} = \frac{\partial E_{o2}}{\partial o_2} \cdot \frac{\partial o_2}{\partial z_4} \cdot \frac{\partial z_4}{\partial h_1} \cdot \frac{\partial h_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_1}$$

https://wikidocs.net/book/2155

손실 함수 (Loss Function) — 예측이 얼마나 틀렸는가

손실 함수(= 비용 함수 = 오차 함수): 예측 값과 실제 값의 차이를 계산하는 함수.
차이가 클수록 손실 값이 크고, 작을수록 손실 값이 작음. 이 값을 최소화하는 가중치(W)와 편향(b)을 찾는 것이 학습의 목표.

1 MSE (평균 제곱 오차)

Mean Squared Error

용도: 연속형 변수 예측 (회귀)

계산: 예측값과 실제값 차이의 제곱을 평균

예: 주가 예측, 온도 예측

$$\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

2 Binary Cross-Entropy

이항 교차 엔트로피

용도: 이진 분류 (2개 클래스)

출력층: Sigmoid 함수 사용

예: 스팸/정상, 양성/음성 분류

$$-\frac{1}{n}\sum_{i=1}^{n}(y_i log\hat{y}_i + (1-y_i)log(1-\hat{y}_i))$$

3 Categorical Cross-Entropy

범주형 교차 엔트로피

용도: 다중 클래스 분류

출력층: Softmax 함수 사용

예: 개/고양이/새 분류,
감정 분류(긍정/부정/중립)

$$-\sum_{i=1}^{n} y_i log(a_i)$$

경사하강법 (Gradient Descent) 과 학습률

경사하강법이란?

손실 함수의 값을 최소화하기 위해
가중치를 조금씩 조정하는 방법

원리:

1. 현재 위치에서 기울기(그래디언트) 계산
2. 기울기의 반대 방향으로 이동
3. 반복하며 최저점(minimum) 탐색

비유:

눈을 감고 산에서 내려가는 것
발밑의 경사를 느끼고, 가장 가파르게 내려가는 방향으로 한 걸음씩 이동

수식:

W(new) = W(old) - learning_rate * dL/dW
(기존 가중치에서 기울기 x 학습률만큼 빼기)

학습률 (Learning Rate)

한 번에 얼마나 이동할지 결정

너무 크면: 최적점을 지나쳐 발산
너무 작으면: 학습 속도가 매우 느림

일반적 값: 1e-5, 5e-5, 1e-4, 1e-3

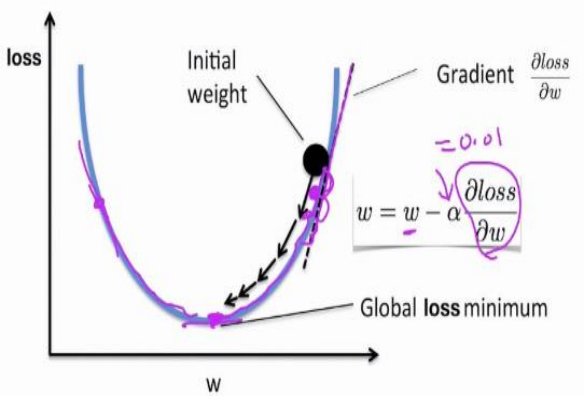
비유: 산을 내려갈 때 보폭의 크기

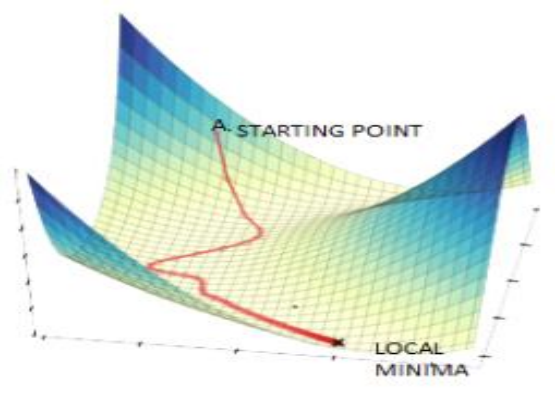
Local vs Global Minimum

Local Minimum:
국소 최저점 (주변에서 가장 낮음)

Global Minimum:
전체에서 가장 낮은 진짜 최저점

경사하강법은 local minimum에
빠질 위험 → 옵티마이저로 해결





https://sanghyu.tistory.com/101

옵티마이저 (Optimizer) — 더 똑똑한 경사하강법

옵티마이저: 손실 함수 값을 최소화하도록 가중치를 조정하는 알고리즘. 단순 경사하강법의 문제(느린 수렴, local minimum)를 개선한 다양한 변형 존재.

1 SGD (Stochastic Gradient Descent)

전체 데이터 대신 미니배치(Mini-Batch) 사용

계산량 감소, 빠른 학습

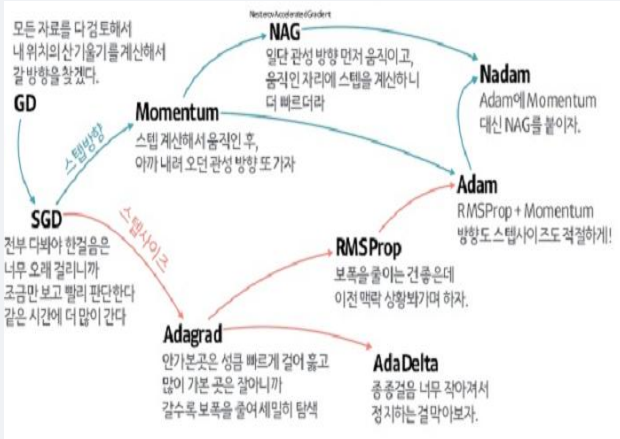
하지만 진동이 심하고 수렴 느릴 수 있음

2 Momentum

이전 그래디언트의 방향성을 유지

관성처럼 같은 방향으로 계속 이동

진동을 줄이고 빠르게 수렴



3 RMSProp

그래디언트의 크기를 정규화

학습률을 자동으로 조정

파라미터마다 다른 학습률 적용

4 Adam (가장 많이 사용)

Momentum + RMSProp 장점 결합

방향성 유지 + 학습률 자동 조정

대부분의 딥러닝에서 기본 선택

needjarvis.tistory.com/685

분류 성능 평가 — 혼동 행렬과 주요 지표

혼동 행렬 (Confusion Matrix)

	예측: 양성	예측: 음성
실제: 양성	TP (True Positive) 맞게 양성 예측	FN (False Negative) 양성을 음성으로
실제: 음성	FP (False Positive) 음성을 양성으로	TN (True Negative) 맞게 음성 예측

TP: 실제 양성을 양성으로 정확히 예측

TN: 실제 음성을 음성으로 정확히 예측

FP: 실제 음성을 양성으로 잘못 예측 (오탐)

FN: 실제 양성을 음성으로 잘못 예측 (미탐)

주요 평가 지표

정확도 (Accuracy)

(TP + TN) / 전체

전체 중 맞힌 비율. 불균형 데이터에서 함정 있음

정밀도 (Precision)

TP / (TP + FP)

양성으로 예측한 것 중 실제 양성 비율

"양성 예측을 얼마나 믿을 수 있나?"

재현율 (Recall)

TP / (TP + FN)

실제 양성 중 얼마나 찾아냈는가

"빠뜨리지 않고 얼마나 잘 찾나?"

F1 Score

2 * (Precision * Recall) / (Precision + Recall)

정밀도와 재현율의 조화 평균

딥러닝의 작동 원리 — 전체 흐름 종합

1 모델 초기화

가중치(W)와 편향(b)을
무작위로 초기화

→ **2 순전파**

입력 데이터가 모든 층을
통과하여 예측값 생성

→ **3 손실 계산**

예측값과 실제값의 차이를
손실 함수로 계산

↓

4 역전파

오차를 이용해 각 가중치의
그래디언트(기울기) 계산

→ **5 가중치 업데이트**

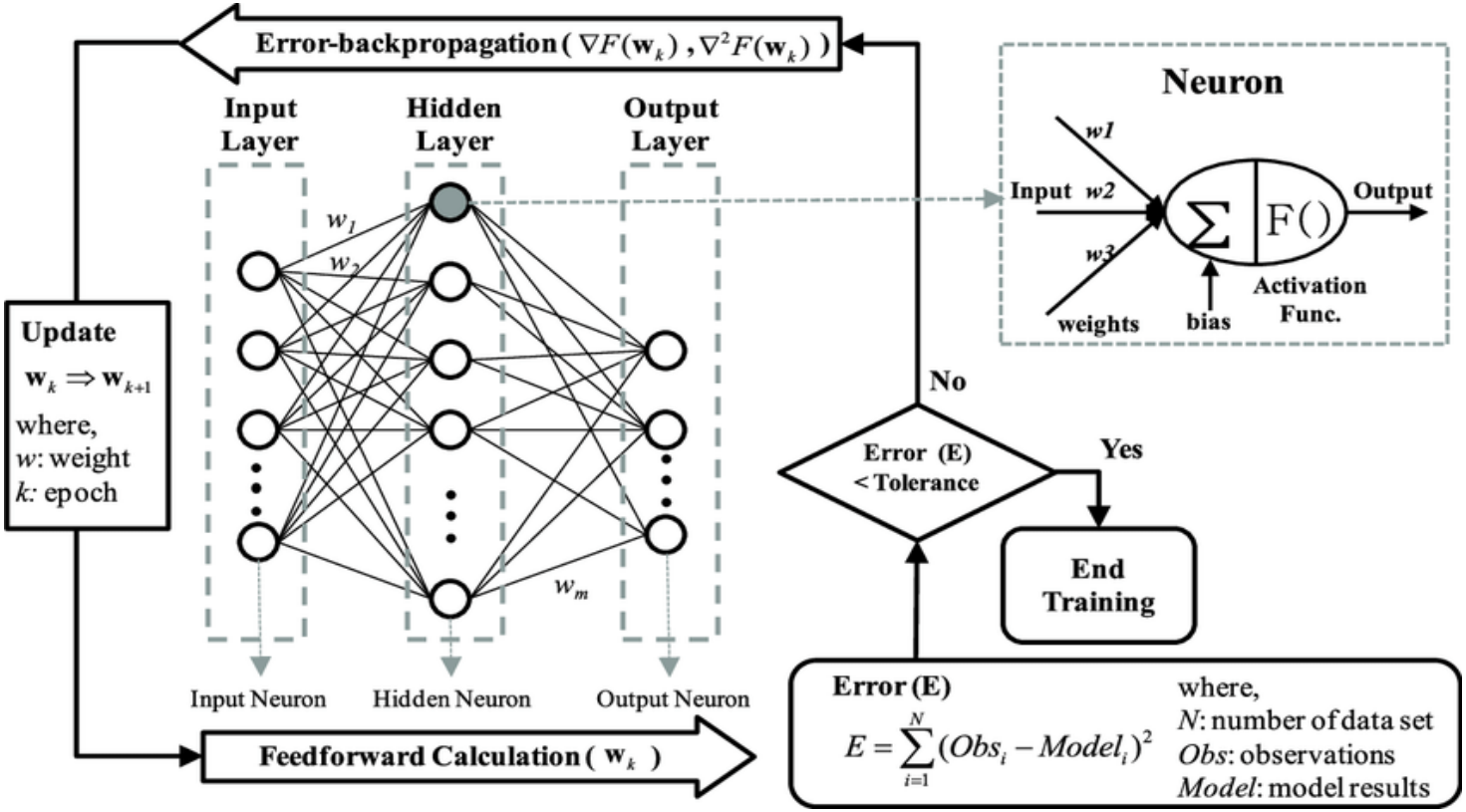
옵티마이저를 사용해
기울기 방향으로 가중치 조정

→ **6 반복**

목표 성능 도달 또는
지정 에포크까지 반복

> 이 과정 전체를 한 줄로 요약하면: "예측하고(순전파) → 틀린 정도를 계산하고(손실) → 어디서 틀렸는지 추적하고(역전파) → 고치는(업데이트) 과정의 반복"

딥러닝의 작동 원리 — 전체 흐름 종합



과적합 (Overfitting) 방지 기법

과적합: 학습 데이터에는 높은 성능을 보이지만 새로운 데이터에는 낮은 성능. 모델이 데이터의 패턴이 아닌 노이즈까지 외워버린 상태. 비유: 기출 문제만 달달 외워서 새 문제를 못 푸는 학생.

1 데이터 양 증가

데이터가 적으면 노이즈까지 학습

데이터 증강(Data Augmentation):

이미지 회전, 반전, 크롭 등으로 확대

2 드롭아웃 (Dropout)

학습 중 일부 뉴런을 무작위 비활성화

특정 뉴런 의존 방지

서로 다른 신경망 앙상블 효과

3 Early Stopping

검증 오차가 증가하기 시작하면

학습을 중단

더 이상 학습해도 성능이 나빠지는 시점을 자동 감지

4 가중치 규제 (Regularization)

L1 규제: 가중치 절대값 합을 손실에 추가

L2 규제: 가중치 제곱합을 손실에 추가

가중치가 너무 커지는 것을 방지

기울기 소실 (Vanishing) 과 폭주 (Exploding) 방지

기울기 소실

역전파 중 그래디언트가 점점 작아져
입력층에 가까운 층의 가중치가
거의 업데이트되지 않는 문제

기울기 폭주

그래디언트가 매우 큰 값으로 발산
가중치가 극단적으로 변해
학습이 불안정해지는 문제

해결 방법

방법	설명	대상
가중치 초기화	균등/정규 분포로 적절히 초기화 (Xavier, He 초기화)	소실+폭주
배치 정규화	활성화 함수 통과 전에 평균/분산으로 정규화 (배치 기준)	소실+폭주
레이어 정규화	하나의 샘플 안에서 모든 뉴런 기준 정규화 (Transformer에서 사용)	소실+폭주
ReLU 계열 사용	시그모이드 대신 ReLU/Leaky ReLU를 은닉층에 사용	소실
잔차 연결	입력값을 다음 층 출력에 직접 더함 (ResNet, Transformer)	소실
그래디언트 클리핑	그래디언트 크기에 상한선 설정	폭주